

LICENCIATURA EN
CIENCIA DE DATOS

PROGRAMACIÓN AVANZADA (189)

Clase 3 — Taller de POO + Git + GitHub

TEMARIO

- T0: **Revisión** de Clase anterior
 - Revisión de dudas
 - Análisis de resoluciones de estudiantes
 - Conclusiones
- T1: Objetivos y Consignas de Práctica Clase N. 3
- T2: Exposición de la Práctica
- T3: Codificación en Python 3
- Temas relacionados
- Links



T0: Revisión de **Clase Anterior**:

- Revisión de dudas
- Análisis de resoluciones de estudiantes
- Conclusiones

Objetivos y Consignas de la clase

- Resolver problemas usando Programación Orientada a Objetos
- Organizar un proyecto de código real
- Aprender flujo básico de Git
- Publicar código en GitHub

¿Cómo vamos a trabajar hoy?

- No vamos a hacer ejercicios sueltos
- Vamos a trabajar como desarrolladores
- Proyecto + código + control de versiones



Flujo de trabajo de la clase

- Crear proyecto
- Inicializar Git
- Subir a GitHub
- Programar
- Guardar cambios (commit)
- Subir avances (push)



Paso 1: Crear el proyecto

Crear carpeta:

```
# Desde la terminal de Windows o Linux
mkdir unab_pa_tp_3
cd unab_pa_tp_3
```

Paso 2: Inicializar Git

```
git init
```

Paso 3: Crear estructura base

Abrimos la carpeta en el editor (ej. VSCode)

```
unab_pa_tp_3/
├── README.md
├── src/
└── main.py
```

```
README.md
# Taller POO - Clase 3

## Integrantes
- Nombre 1
- Nombre 2

## Descripción
Resolución de ejercicios de POO con Git

## Ejercicios
- Punto
- Linea
- Cancion
- Libro
```

Primer commit (muy importante)

```
git add .  
git commit -m "Estructura inicial  
del proyecto"
```

Con esto guardamos el estado inicial 

Paso 4: Crear repo en GitHub

- Ir a GitHub
- New repository
- Nombre: unab_pa_tp_3

Paso 5: Conectar con GitHub

```
git remote add origin URL  
git branch -M main  
git push -u origin main
```

Proyecto ya publicado 

Buenas prácticas

- Commits pequeños y claros
- No acumular cambios
- Subir seguido (push)
- Código ordenado

Cada vez que avanzamos:

- Escribimos código
- Guardamos cambios
- Los subimos

Flujo de trabajo real

```
git add .  
git commit -m "mensaje claro"  
git push
```

Nivel 1: Clase Punto

- Crear archivo `punto.py` dentro de `src`
- Crear la clase ***Punto*** con dos atributos `x` e `y` (ambos numéricos), con el correspondiente constructor que recibe ambos valores.
- Definir métodos tales como:
 - `eje_x`, `eje_y`
 - `impresion` (devuelve en representación de string ambos valores)
 - `opuesto` (devuelve el punto opuesto)
 - Cualquier otro método que considere importante

Después de terminar la clase Punto:

```
git add .  
git commit -m "Agrego clase Punto"  
git push
```


Nivel 2: Clase Linea

Define una clase **Línea** con dos atributos: `_punto_a` y `_punto_b`. Son dos puntos por los que pasa la línea en un espacio de dos dimensiones. Crear archivo `linea.py` dentro de `src`

La clase dispondrá de los siguientes métodos:

- **Linea(Punto, Punto)** Constructor que recibe como parámetros dos objetos de la clase **Punto**, que son utilizados para inicializar los atributos.
- **mueve_derecha(float)** Desplaza la línea a la derecha la distancia que se indique.
- **mueve_izquierda(float)** Desplaza la línea a la izquierda la distancia que se indique.
- **mueve_arriba(float)** Desplaza la línea hacia arriba la distancia que se indique.
- **mueve_abajo(float)** Desplaza la línea hacia abajo la distancia que se indique.

Después de terminar la clase Linea

```
git add .  
git commit -m "Agrego clase Linea"  
git push
```

Nivel 3: Clase Cancion

Crear archivo `cancion.py` dentro de `src` y desarrollar la clase ***Cancion*** con los siguientes atributos:

- `titulo`: una variable `String` que guarda el título de la canción.
- `autor`: una variable `String` que guarda el autor de la canción.

Con los siguientes métodos:

- **`Cancion(String, String)`**: constructor que recibe como parámetros el título y el autor de la canción (por este orden).
- **`get_titulo()`**: devuelve el título de la canción.
- **`get_autor()`**: devuelve el autor de la canción.
- **`set_titulo(String)`**: establece el título de la canción.
- **`set_autor(String)`**: establece el autor de la canción

```
# Al terminar  
git add .  
git commit -m "Agrego clase Cancion"  
git push
```

Nivel 4: Clase Libro

- Crea una clase Libro que modele la información que se mantiene en una biblioteca sobre cada libro: título, autor (usa la clase Persona), ISBN, páginas, edición, editorial , lugar (ciudad y país) y fecha de edición (como texto). La clase debe proporcionar los siguientes servicios: getters y setters, método para leer la información y método para mostrar la información. Crear archivo `libro.py` dentro de `src`
- Este último método mostrará la información del libro con este formato:

Título: Introduction to Java Programming 3a. edición

Autor: Liang, Y. Daniel

ISBN: 0-13-031997-X

Prentice-Hall, New Jersey (USA)

viernes 16 de noviembre de 2001

784 páginas

```
# Al terminar
git add .
git commit -m "Agrego clase Libro"
git push
```

Checklist final:

- Repo creado
- Código subido
- Múltiples commits
- README completo

Composición

- Relación **fuerte** entre objetos
- Un objeto está **formado por otros** que son parte esencial
- Los objetos internos **no existen sin el principal**
- Comparten el mismo **ciclo de vida**

Ejemplo:

Auto → Motor, Ruedas

Si el auto desaparece, sus partes también

Diferencia clave

- **Composición** → dependencia total (vida ligada)
- **Agregación** → independencia (vida separada)

Agregación

- Relación **débil** entre objetos
- Un objeto **contiene o usa** a otros
- Los objetos internos **pueden existir por separado**
- **No comparten** necesariamente el ciclo de vida

Ejemplo:

Equipo → Jugadores

El equipo puede desaparecer, pero los jugadores siguen existiendo

Link de referencia: <https://juncotic.com/poo-composicion-y-agregacion/>

Links interesantes:

Aprenda a pensar como un programado con Python

<https://argentinaenpython.com/quiero-aprender-python/aprenda-a-pensar-como-un-programador-con-python.pdf>

Documentación oficial:

<https://www.python.org/doc/>

Curso de Python desde 0 (pildorasinformaticas)

<https://www.youtube.com/playlist?list=PLU8oAIHdN5BlvPxziopYZRd55pdqFwkeS>

Otros links:

<https://ellibrodepython.com/polimorfismo-en-programacion>

<https://parzibyte.me/blog/2019/06/30/clases-constructores-python-poo/>

¡Muchas gracias por tu atención!